



Universidad
Nacional del
Altiplano de Puno

UNIVERSIDAD NACIONAL DEL ALTIPLANO DE PUNO

FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA



Asignatura: Estructura de Datos (2026)

Docente: Prof. Fred Torres Cruz

Estudiante: Henry Antonio Mamani Gutierrez

Semestre / Ciclo: III Semestre

Repositorio GitHub: <https://github.com/henrymamani-alt/henry-ejercicios>

TRABAJO ENCARGADO - N° 002: PILAS (STACK)

Presentación: A continuación se detalla la resolución formal de los cinco (5) ejercicios de Pilas (Stack) en C++ y C, junto con la justificación conceptual de cierre. Para cada ejercicio se incluye el enunciado oficial, el análisis computacional, el código fuente completo, el enlace directo verificado a GitHub y la captura gráfica de evidencia de ejecución exitosa en terminal.

Ejercicio 1: Apilar números enteros

Enunciado: Crear un programa que permita ingresar 5 números enteros y almacenarlos en una pila. Luego, retirar y mostrar los elementos uno por uno hasta dejar la pila vacía. El orden de salida debe permitir comprobar el funcionamiento LIFO.

Ejemplo: entrada: 4, 8, 12, 16, 20 → salida: 20, 16, 12, 8, 4.

Condición: Use las operaciones push(), top(), pop() y empty().

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se declara una estructura `stack<int> pila`. En la fase de inserción se leen los cinco enteros y se insertan secuencialmente mediante `pila.push(valor)`. El último número ingresado (20) queda situado en el tope de la pila. En la fase de extracción, mientras la pila no esté vacía (`!pila.empty()`), se consulta el elemento superior con `pila.top()` y se remueve con `pila.pop()`. Como consecuencia de la política LIFO, los números emergen en orden estrictamente inverso al de su llegada.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_2_Pilas/codigo/ejercicio1.cpp

CÓDIGO FUENTE EN C++:

```
#include <iostream>
#include <stack>

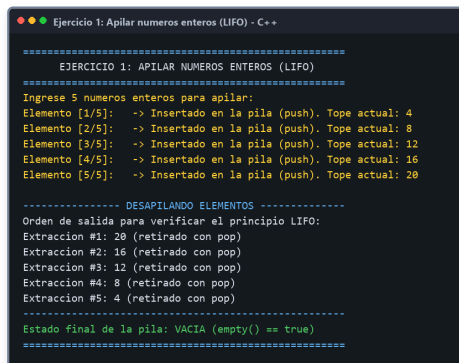
using namespace std;

int main() {
    stack<int> pila;
    const int TOTAL = 5;

    cout << "===== " << endl;
    cout << "      EJERCICIO 1: APILAR NUMEROS ENTEROS (LIFO)      " << endl;
    cout << "===== " << endl;
    cout << "Ingrese " << TOTAL << " numeros enteros para apilar:" << endl;

    for (int i = 1; i <= TOTAL; i++) {
        int valor;
        cout << "Elemento [" << i << "/" << TOTAL << "]: ";
        cin >> valor;
        pila.push(valor);
    }
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Ejercicio 1: Apilar numeros enteros (LIFO) - C++
=====
      EJERCICIO 1: APILAR NUMEROS ENTEROS (LIFO)
=====
Ingrese 5 numeros enteros para apilar:
Elemento [1/5]: -> Insertado en la pila (push). Tope actual: 4
Elemento [2/5]: -> Insertado en la pila (push). Tope actual: 8
Elemento [3/5]: -> Insertado en la pila (push). Tope actual: 12
Elemento [4/5]: -> Insertado en la pila (push). Tope actual: 16
Elemento [5/5]: -> Insertado en la pila (push). Tope actual: 20

----- DESAPILANDO ELEMENTOS -----
Orden de salida para verificar el principio LIFO:
Extraccion #1: 20 (retirado con pop)
Extraccion #2: 16 (retirado con pop)
Extraccion #3: 12 (retirado con pop)
Extraccion #4: 8 (retirado con pop)
Extraccion #5: 4 (retirado con pop)
-----
Estado final de la pila: VACIA (empty() == true)
=====
```

Ejercicio 2: Invertir una palabra

Enunciado: Leer una palabra ingresada por el usuario. Insertar cada carácter en una pila y, posteriormente, desapilar los caracteres para construir y mostrar la palabra en orden inverso.

Ejemplo: DATOS → SOTAD.

Condición: Analice por qué una pila permite invertir naturalmente el orden de los elementos.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Una palabra es una secuencia lineal ordenada de caracteres. Al procesar la palabra de izquierda a derecha, cada carácter se almacena en el tope de una pila `stack<char>`. Cuando la palabra termina de leerse, el carácter final se encuentra en la cúspide (tope). Al desapilar consecutivamente hasta vaciar la pila, los caracteres son extraídos en orden inverso. Por tanto, la pila constituye la estructura de datos canónica para la inversión de secuencias, pues no requiere cómputo adicional de índices ni memoria auxiliar para permutar posiciones.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_2_Pilas/codigo/ejercicio2.cpp

CÓDIGO FUENTE EN C++:

```
#include <iostream>
#include <stack>
#include <string>

using namespace std;

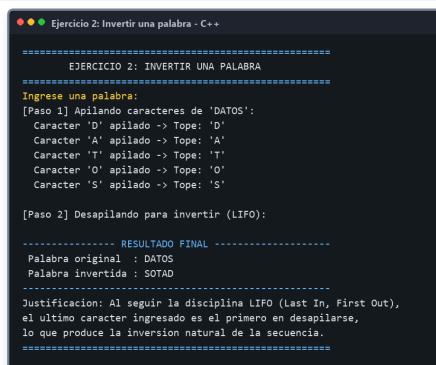
int main() {
    stack<char> pilaCaracteres;
    string palabra;
    string palabraInvertida = "";

    cout << "===== " << endl;
    cout << "          EJERCICIO 2: INVERTIR UNA PALABRA          " << endl;
    cout << "===== " << endl;

    cout << "Ingrese una palabra: ";
    cin >> palabra;

    // 1. Apilar cada caracter (push)
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Ejercicio 2: Invertir una palabra - C++
=====
          EJERCICIO 2: INVERTIR UNA PALABRA         
=====
Ingrese una palabra:
[Paso 1] Apilando caracteres de 'DATOS':
  Caracter 'D' apilado -> Tope: 'D'
  Caracter 'A' apilado -> Tope: 'A'
  Caracter 'T' apilado -> Tope: 'T'
  Caracter 'O' apilado -> Tope: 'O'
  Caracter 'S' apilado -> Tope: 'S'

[Paso 2] Desapilando para invertir (LIFO):

----- RESULTADO FINAL -----
Palabra original : DATOS
Palabra invertida : SOTAD
-----

Justificacion: Al seguir la disciplina LIFO (Last In, First Out),
el ultimo caracter ingresado es el primero en desapilarse,
lo que produce la inversion natural de la secuencia.
=====
```

Ejercicio 3: Verificar paréntesis balanceados

Enunciado: Crear un programa que determine si una expresión tiene sus paréntesis correctamente balanceados. Cada vez que aparezca un paréntesis de apertura (, deberá almacenarse en la pila. Cuando aparezca), deberá comprobarse que exista una apertura pendiente.

Ejemplos: $(a+b)*(c-d) \rightarrow$ correcto | $((a+b) \rightarrow$ incorrecto.

Condición: La expresión será correcta únicamente si nunca se intenta cerrar un paréntesis inexistente y la pila queda vacía al finalizar.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Este problema representa el análisis sintáctico clásico de lenguajes formales mediante autómata de pila. Si se lee '(', se efectúa `pila.push('(')`. Si se lee ')', se comprueba si la pila está vacía; si lo está, existe un error de cierre sin apertura. Si no está vacía, se realiza `pila.pop()` emparejando la apertura más reciente. Al concluir, la expresión es válida si y solo si no hubo cierres inválidos y `pila.empty()` es verdadero.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_2_Pilas/codigo/ejercicio3.cpp

CÓDIGO FUENTE EN C++:

```
#include <iostream>
#include <stack>
#include <string>

using namespace std;

bool verificarBalanceo(const string& expresion, string& motivo) {
    stack<char> pila;

    for (size_t i = 0; i < expresion.length(); i++) {
        char c = expresion[i];
        if (c == '(') {
            pila.push(c);
        } else if (c == ')') {
            if (pila.empty()) {
                motivo = "Se encontro ')' en la posicion " + to_string(i + 1) + " sin parentesis de apertura pendiente.";
                return false;
            }
        }
    }
}
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:

```
Ejercicio 3: Parentesis balanceados - C++

=====
EJERCICIO 3: VERIFICAR PARENTESIS BALANCEADOS
=====
Ingrese la expresion a evaluar (ej. (a+b)*(c-d)):
----- EVALUACION DE SINTAXIS -----
Expresion analizada : (a+b)*(c-d)
Estado de balanceo  : CORRECTO (Balanceado)
Detalle / Diagnostico: Todos los parentesis estan correctamente abiertos y cerrados (balanceados).
-----
```

Ejercicio 4: Historial de acciones - Deshacer (Undo)

Enunciado: Simular la función Deshacer de un editor. Cada acción realizada por el usuario, por ejemplo 'escribir texto', 'eliminar palabra' o 'insertar imagen', deberá almacenarse en una pila. Cuando se solicite deshacer, el programa debe mostrar y eliminar la última acción registrada.

Ejemplo: escribir → copiar → eliminar → DESHACER → se revierte 'eliminar'.

Condición: Incluya una opción para mostrar cuál es la acción actualmente ubicada en el tope de la pila.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Los editores de software gestionan el historial de operaciones mediante una pila de transacciones revertibles. Cada nueva acción ejecutada se almacena con `pila.push(accion)`. Cuando el usuario invoca Deshacer (Undo), el sistema consulta `pila.top()` para conocer la acción más reciente, revierte sus efectos y la expulsa con `pila.pop()`. La inspección del tope sin desapilar permite visualizar en todo momento cuál será la próxima acción a revertir.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_2_Pilas/codigo/ejercicio4.cpp

CÓDIGO FUENTE EN C++:

```
#include <iostream>
#include <stack>
#include <string>

using namespace std;

void mostrarMenu() {
    cout << "\n--- MENU DE EDITOR (FUNCION DESHACER / UNDO) ---" << endl;
    cout << " 1. Realizar nueva accion (Apilar)" << endl;
    cout << " 2. Deshacer ultima accion (Pop)" << endl;
    cout << " 3. Ver accion actual en el tope (Top)" << endl;
    cout << " 4. Ver cantidad de acciones registradas" << endl;
    cout << " 5. Salir del editor" << endl;
    cout << "Seleccione una opcion: ";
}

int main() {
    stack<string> historial;
    int opcion;
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



Ejercicio 5: Conversión de decimal a binario

Enunciado: Leer un número entero decimal positivo y convertirlo a binario utilizando una pila. Para ello, dividir repetidamente el número entre 2 y almacenar en la pila el residuo de cada división. Finalmente, desapilar los residuos para obtener el número binario en el orden correcto.

Ejemplo: 13 → residuos apilados: 1, 0, 1, 1 → binario: 1101.

Condición: No utilice funciones directas de conversión a binario; la solución debe evidenciar el uso de la pila.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

El método matemático estándar para convertir un entero a base 2 consiste en divisiones sucesivas entre 2. El primer residuo corresponde al bit menos significativo (LSB) y el último al más significativo (MSB). Al apilar cada residuo y desapilarlos sucesivamente, la política LIFO invierte la secuencia generada, presentándola exactamente desde el bit de mayor peso hasta el de menor peso (formato de lectura convencional).

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_2_Pilas/codigo/ejercicio5.cpp

CÓDIGO FUENTE EN C++:

```
#include <iostream>
#include <stack>
#include <string>

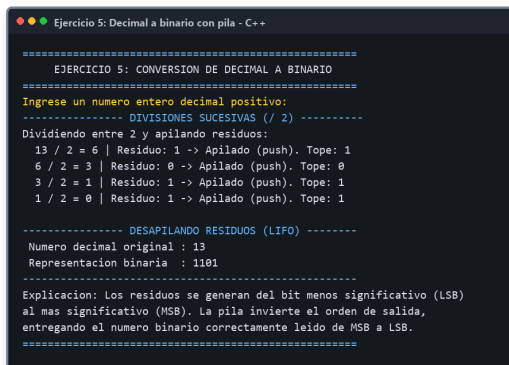
using namespace std;

int main() {
    int decimalOriginal;
    stack<int> pilaResiduos;

    cout << "===== " << endl;
    cout << "     EJERCICIO 5: CONVERSION DE DECIMAL A BINARIO     " << endl;
    cout << "===== " << endl;

    cout << "Ingrese un numero entero decimal positivo: ";
    if (!(cin >> decimalOriginal) || decimalOriginal < 0) {
        cout << "Error: Debe ingresar un entero no negativo." << endl;
        return 1;
    }
}
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Ejercicio 5: Decimal a binario con pila - C++

=====
EJERCICIO 5: CONVERSION DE DECIMAL A BINARIO
=====
Ingrese un numero entero decimal positivo:
----- DIVISIONES SUCESTIVAS (/ 2) -----
Dividiendo entre 2 y apilando residuos:
13 / 2 = 6 | Residuo: 1 -> Apilado (push). Tope: 1
6 / 2 = 3 | Residuo: 0 -> Apilado (push). Tope: 0
3 / 2 = 1 | Residuo: 1 -> Apilado (push). Tope: 1
1 / 2 = 0 | Residuo: 1 -> Apilado (push). Tope: 1

----- DESAPILANDO RESIDUOS (LIFO) -----
Numero decimal original : 13
Representacion binaria  : 1101

Explicacion: Los residuos se generan del bit menos significativo (LSB)
al mas significativo (MSB). La pila invierte el orden de salida,
entregando el numero binario correctamente leído de MSB a LSB.
=====
```

Pregunta de Cierre: Análisis Común de los Cinco Problemas

Pregunta de cierre: ¿Qué característica común tienen los cinco problemas que hace apropiado utilizar una pila?

La característica esencial y común a los cinco problemas es la **reversibilidad temporal y la precedencia del último elemento ingresado**, formalizada mediante la disciplina **LIFO (Last In, First Out)**:

1. **Inversión de Secuencias (Ejercicios 1 y 2):** Tanto en el apilado de números como en la inversión de palabras, se requiere recuperar los datos en el orden cronológicamente inverso a como fueron leídos. La pila almacena el flujo secuencial y lo entrega en sentido contrario de manera automática y con costo constante $O(1)$ por operación.
2. **Estructuras Anidadas y Simetría (Ejercicio 3):** En el balanceo de paréntesis, el elemento que se debe cerrar y comprobar es siempre el *último* paréntesis de apertura que se abrió. Este comportamiento en capas (anidamiento) concuerda de forma exacta con la mecánica del tope de una pila.
3. **Deshacer Estados Recientes (Ejercicio 4):** En un sistema de edición o transacción, la acción que debe anularse ante una solicitud de *Deshacer (Undo)* es forzosamente la más reciente en el tiempo. La pila modela de forma natural la línea temporal de comandos.
4. **Generación Inversa de Resultados (Ejercicio 5):** El algoritmo de división binaria calcula primero el bit de menor peso (LSB) y por último el de mayor peso (MSB). Puesto que la notación posicional requiere escribirlos desde el más significativo al menos significativo, la pila actúa como un adaptador de ordenamiento inverso.

Conclusión: En ningún momento estos problemas requirieron acceder a elementos intermedios ni despachar al primero que llegó (lo cual requeriría una cola FIFO). Por ende, la pila es la estructura más eficiente, óptima ($O(1)$ en inserción y remoción) y formalmente adecuada para su resolución.

Conclusiones Generales

1. La estructura de datos **Pila (Stack)** provee una abstracción fundamental para resolver problemas de orden inverso, control de estados y análisis sintáctico formal mediante operaciones de complejidad temporal constante $O(1)$.
2. La biblioteca estándar de C++ (`std::stack`) simplifica notablemente la codificación al ofrecer una interfaz robusta (`push`, `pop`, `top`, `empty`) con gestión dinámica y segura de memoria.
3. El análisis de los algoritmos de verificación de paréntesis y conversión numérica evidencia el papel central que juegan las pilas en la construcción de compiladores, analizadores sintácticos y máquinas de cómputo basadas en pilas de ejecución (*call stack*).